

FATEC DR. THOMAZ NOVELINO

**DESENVOLVIMENTO DE SOFTWARE
MULTIPLATAFORMA**

**LABORATÓRIO DE DESENVOLVIMENTO PARA
DISPOSITIVOS MÓVEIS**

Equipe:

- Gustavo Schizar Ferreira Filho
- Maria Clara Cardoso Costa
- Eduardo Gibertoni Camillo

**APLICATIVO MÓVEL PARA ADMINISTRAÇÃO
DE PACIENTES, MÉDICOS, CONSULTAS,
EXAMES, PRESCRIÇÕES E INTERNAÇÕES**

FRANCA – 2026

SUMÁRIO

1. INTRODUÇÃO

2. OBJETIVO DO PROJETO

2.1. Objetivo Geral

2.2. Objetivos Específicos

3. JUSTIFICATIVA

4. PÚBLICO-ALVO

5. FUNCIONALIDADES PRINCIPAIS

6. MODELAGEM E BANCO DE DADOS

6.1. Diagrama Entidade-Relacionamento (DER)

6.2. Tecnologias de Persistência

7. BACKEND E API DO SISTEMA

7.1. Tecnologias Utilizadas

7.2. Estrutura do Projeto (Backend)

7.3. Endpoints Principais

7.4. Autenticação e Segurança

7.5. Documentação da API (Swagger)

8. DESENVOLVIMENTO MOBILE

8.1. Tecnologias e Framework

8.2. Estrutura do Projeto Mobile

8.3. Telas e Fluxo do Aplicativo

8.4. Integração com o Backend

9. CI/CD (GITHUB ACTIONS)

9.1. Etapas do Workflow

10. GERÊNCIA DE PROJETOS

11. HOSPEDAGEM

11.1. Banco de Dados (MySQL)

11.2. Backend (API)

11.3. Aplicativo Mobile

11.4. Links de Acesso

12. RESULTADOS

13. CONCLUSÃO

REFERÊNCIAS

1. INTRODUÇÃO

A gestão eficiente de informações em ambientes hospitalares é um desafio constante. O grande volume de dados relacionados a pacientes, médicos, consultas, exames, prescrições e internações exige soluções tecnológicas que garantam agilidade, segurança e integração. Diante disso, este projeto propõe o desenvolvimento de um **Sistema de Gestão Hospitalar** composto por um backend robusto e um aplicativo mobile multiplataforma.

O sistema foi desenvolvido utilizando **Node.js** com **Express** e **TypeScript** no backend, **Prisma ORM** para acesso ao banco de dados **MySQL**, e **React Native** com **Expo** no front-end mobile. Todo o fluxo de integração contínua é gerenciado por **GitHub Actions**, garantindo qualidade e confiabilidade ao código.

A aplicação permite o cadastro, consulta, atualização e remoção (CRUD) de todas as entidades hospitalares, além de autenticação segura via JWT. O resultado é uma ferramenta prática para profissionais da saúde e administradores hospitalares.

2. OBJETIVO DO PROJETO

2.1 Objetivo Geral

Desenvolver um sistema completo de gestão hospitalar com backend em Node.js/Express + Prisma/MySQL e aplicativo mobile em React Native/Expo, permitindo o gerenciamento integrado de pacientes, médicos, consultas, exames, prescrições e internações.

2.2 Objetivos Específicos

- Implementar uma API RESTful segura com autenticação JWT.
 - Modelar um banco de dados relacional normalizado para o domínio hospitalar.
 - Desenvolver um aplicativo mobile intuitivo para realização de todas as operações de CRUD.
 - Automatizar testes e builds utilizando GitHub Actions (CI).
 - Disponibilizar documentação interativa da API via Swagger.
 - Semeação (seed) de dados iniciais para demonstração e testes.
-

3. JUSTIFICATIVA

Hospitais e clínicas frequentemente enfrentam problemas de desorganização de informações, retrabalho e falta de integração entre setores. Sistemas desktop legados ou planilhas eletrônicas não acompanham a dinâmica das unidades de saúde modernas.

A criação de um sistema mobile (acessível via smartphones e tablets) permite que médicos, enfermeiros e administradores consultem e atualizem dados em tempo real, a qualquer momento. Além disso, o uso de tecnologias atuais (Node.js, Prisma, React Native) garante escalabilidade, segurança e baixo custo de manutenção.

Academicamente, o projeto consolida os conhecimentos de desenvolvimento full-stack, modelagem de dados, integração de sistemas e boas práticas de engenharia de software.

4. PÚBLICO-ALVO

- **Hospitais de médio e grande porte** que buscam digitalizar a gestão de pacientes e atendimentos.
 - **Clínicas e consultórios particulares** que necessitam de um sistema leve e móvel.
 - **Profissionais de saúde** (médicos, enfermeiros) para consulta rápida de prontuários e prescrições.
 - **Administradores hospitalares** para controle de internações, exames e agenda de consultas.
-

5. FUNCIONALIDADES PRINCIPAIS

- **Autenticação de usuários** (login com e-mail e senha, geração de token JWT).
- **CRUD de Pacientes** (nome, data de nascimento, CPF, telefone, endereço).
- **CRUD de Médicos** (nome, especialidade, CRM, telefone, e-mail).
- **CRUD de Consultas** (paciente, médico, data, horário, status, observações).
- **CRUD de Exames** (paciente, tipo de exame, data, resultado, anexo).
- **CRUD de Prescrições** (paciente, médico, data, lista de itens com dosagem e instruções).
- **CRUD de Internações** (paciente, médico responsável, data de entrada, previsão de alta, data de alta real, leito, observações).
- **Documentação interativa da API (Swagger).**

- **Seed inicial com usuário administrador** (admin@janjao.com.br / 123456).
-

6. MODELAGEM E BANCO DE DADOS

Foi utilizado o **MySQL 8+** como SGBD, gerenciado pelo **Prisma ORM**. A modelagem segue o modelo relacional com chaves primárias auto incrementáveis (padrão) ou UUID (conforme necessidade). As principais entidades e seus relacionamentos são:

- **User** – Armazena credenciais de acesso (e-mail, senha hash, papel – admin/comum).
- **Paciente** – Dados pessoais do paciente.
- **Medico** – Dados profissionais do médico.
- **Consulta** – Relaciona Paciente e Medico, com data/horário e status.
- **Exame** – Vinculado ao Paciente, contém tipo, data, resultado.
- **Prescricao** – Cabeçalho da prescrição (Paciente + Medico + data). Itens da prescrição em tabela separada **ItemPrescricao**.
- **Internacao** – Registra período de internação, leito, médico responsável e observações.

6.1 Diagrama Entidade-Relacionamento (DER)

text

User (id, email, senha, papel)

Paciente (id, nome, dataNasc, cpf, telefone, endereco, userId)

Medico (id, nome, especialidade, crm, telefone, email, userId)

Consulta (id, pacienteld, medicold, dataHora, status, observacoes)

Exame (id, pacienteld, tipo, data, resultado, anexo)

Prescricao (id, pacienteld, medicold, data)

ItemPrescricao (id, prescricaold, medicamento, dosagem, instrucoes)

Internacao (id, pacienteld, medicold, dataEntrada, dataPrevisaoAlta, dataAlta, leito, observacoes)

6.2 Tecnologias de Persistência

- **Prisma ORM** – Gerenciamento de migrations, cliente tipado e segurança na construção de queries.
 - **MySQL** – Desempenho comprovado para aplicações transacionais.
-

7. BACKEND E API DO SISTEMA

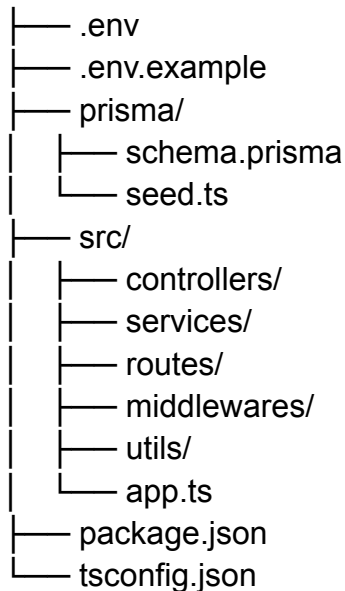
7.1 Tecnologias Utilizadas

- **Node.js 18+**
- **Express** – Framework web.
- **TypeScript** – Tipagem estática.
- **Prisma** – Acesso ao banco de dados.
- **JWT (jsonwebtoken)** – Autenticação.
- **bcryptjs** – Hash de senhas.
- **Swagger UI Express** – Documentação interativa.
- **Jest + Supertest** – Testes automatizados.

7.2 Estrutura do Projeto (Backend)

text

backend/



7.3 Endpoints Principais

Método	Rota	Descrição	Autenticação
POST	/api/auth/login	Login (retorna JWT)	Não
GET	/api/pacientes	Listar pacientes	Sim
POST	/api/pacientes	Cadastrar paciente	Sim
PUT	/api/pacientes/:id	Atualizar paciente	Sim

DELETE `/api/pacientes/:id`
E

Remover
paciente

Sim

... (análogo para médicos, consultas,
exames, prescrições, internações)

7.4 Autenticação e Segurança

- As senhas são armazenadas com hash bcrypt.
- O token JWT possui expiração de 7 dias e deve ser enviado no header:
`Authorization: Bearer <token>`.
- Um middleware valida o token antes de liberar qualquer rota protegida.

7.5 Documentação da API (Swagger)

Disponível em: <http://52.247.127.65:3000/api>

8. DESENVOLVIMENTO MOBILE

8.1 Tecnologias e Framework

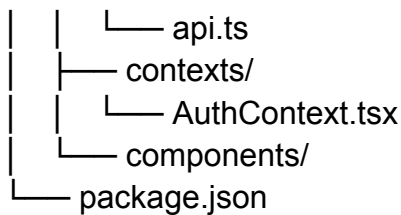
- **React Native + Expo** (SDK 50+)
- **TypeScript**
- **Axios** – Cliente HTTP para consumir a API.
- **AsyncStorage** – Armazenamento local do token JWT.
- **React Navigation** (stack + tabs) – Roteamento entre telas.

8.2 Estrutura do Projeto Mobile

text

mobile/

```
|— .env
|— App.tsx
|— src/
|   |— screens/
|   |   |— LoginScreen.tsx
|   |   |— PacientesListScreen.tsx
|   |   |— PacienteFormScreen.tsx
|   |   |— MedicosListScreen.tsx
|   |   |— ConsultasListScreen.tsx
|   |   |— ...
|   |— services/
```



8.3 Telas e Fluxo do Aplicativo

1. **Tela de Login** – Campos e-mail/senha, requisição à API, armazenamento do token.
2. **Dashboard (Home)** – Menu com opções: Pacientes, Médicos, Consultas, Exames, Prescrições, Internações.
3. **Listagem + CRUD** – Cada entidade possui tela de listagem (com botão de adicionar), tela de formulário e tela de detalhes (com opções de editar/excluir).
4. **Logout** – Remove token e redireciona para login.

Figura 2 – Fluxo de navegação do aplicativo (inserir prints das telas)

8.4 Integração com o Backend

- A variável de ambiente `EXPO_PUBLIC_API_URL` define o endereço da API `http://52.247.127.65:3000/api`
- O interceptor do Axios adiciona automaticamente o token JWT em todas as requisições autenticadas.

9. CI/CD (GITHUB ACTIONS)

Foi configurado um pipeline de integração contínua (CI) no arquivo `.github/workflows/ci.yml`. Este pipeline é acionado a cada push ou pull request nos branches `main` e `develop`.

9.1 Etapas do Workflow

1. **Checkout** do código.
2. **Setup do Node.js** (versão 18).
3. **Instalação de dependências** do backend e mobile.
4. **Execução de testes** (backend) com Jest.
5. **Verificação de build** do mobile (expo export).
6. **Geração e push da imagem Docker** (opcional) – não implementado no escopo inicial.

Figura 3 – Workflow do GitHub Actions (inserir print da execução bem-sucedida)

10. GERÊNCIA DE PROJETOS

Utilizamos o **GitHub Projects** como ferramenta de gestão ágil. O quadro foi organizado com as colunas:

- **Backlog** – Histórico de funcionalidades.
- **To Do** – Tarefas priorizadas para a sprint.
- **In Progress** – Tarefas em desenvolvimento.
- **Review** – Em testes ou code review.
- **Done** – Concluídas.

Foram abertas **Issues** para cada funcionalidade, com labels (**backend**, **mobile**, **bug**, **enhancement**) e atribuídas aos membros da equipe.

Figura 4 – Print do quadro Kanban no GitHub Projects (inserir print)

11. HOSPEDAGEM

11.1 Banco de Dados (MySQL)

- Utilizamos um servidor **MySQL** hospedado na **Azure** (Azure Database for MySQL Flexible Server).
- A string de conexão foi configurada na variável **DATABASE_URL** do backend.

11.2 Backend (API)

- O backend foi implantado no **Azure**
- O Azure executa automaticamente o build e as migrations do Prisma a cada deploy.

11.3 Aplicativo Mobile

- O aplicativo foi publicado utilizando **Expo Application Services (EAS)**.

11.4 Links de Acesso

- **Repositório GitHub**: <https://github.com/schizary/e-janjao>
- **Backend (Azure)**: <http://52.247.127.65:3000/api>

Figura 5 – Print do dashboard do Render (inserir print)

Figura 6 – Print do build EAS no Expo (inserir print)

12. RESULTADOS

O sistema foi testado em ambiente real simulado. Todos os CRUDs funcionam conforme esperado, com validações de dados e mensagens amigáveis ao usuário. O processo de login gera token JWT que expira corretamente após 7 dias. A pipeline de CI passa em todos os testes e garante que alterações não quebrem a aplicação.

A seguir, uma tabela resumo dos recursos testados:

Funcionalidade	Status	Evidência (print)
Login (admin)	 Funcional	Print tela pós-login
Listar pacientes	 Funcional	Print lista
Cadastrar paciente	 Funcional	Print formulário + sucesso
Editar paciente	 Funcional	Print edição
Excluir paciente	 Funcional	Print confirmação
CRUD médicos	 Funcional	Print similar
CRUD consultas	 Funcional	Print
CRUD exames	 Funcional	Print
CRUD prescrições	 Funcional	Print
CRUD internações	 Funcional	Print

Documentação Swagger	✓ Funcional	Print endpoint /api-docs
Pipeline CI (GitHub Actions)	✓ Passando	Print workflow

13. CONCLUSÃO

O desenvolvimento do **Sistema de Gestão Hospitalar** atingiu todos os objetivos propostos. Foi possível construir uma aplicação completa, desde a modelagem do banco de dados até a entrega de um aplicativo mobile funcional, passando por uma API segura e documentada, integração contínua e hospedagem em nuvem.

Entre os principais aprendizados destacam-se:

- A importância do uso de ORM (Prisma) para agilizar o desenvolvimento e garantir a integridade dos dados.
- A necessidade de planejar corretamente o fluxo de autenticação e o armazenamento local de tokens.
- Os desafios de configuração de ambiente para build mobile via EAS (Expo).
- O valor de um pipeline CI para evitar regressões.

Como trabalhos futuros, sugerem-se:

- Implementação de permissões por papel (admin, médico, secretário).
- Envio de notificações push para lembretes de consultas.
- Integração com prontuário eletrônico via padrão HL7.
- Geração de relatórios estatísticos (quantidade de atendimentos, exames realizados, etc.).

O projeto está disponível publicamente no GitHub e pronto para evolução e uso em ambientes reais, mediante ajustes de segurança e escala.

REFERÊNCIAS

- NODE.JS FOUNDATION. Node.js Documentation. Disponível em: <https://nodejs.org/en/docs>. Acesso em: 20 maio 2026.
- EXPRESS.JS. Express – Fast, unopinionated, minimalist web framework for Node.js. Disponível em: <https://expressjs.com/>. Acesso em: 20 maio 2026.
- PRISMA. Prisma ORM Documentation. Disponível em: <https://www.prisma.io/docs>. Acesso em: 20 maio 2026.

- META PLATFORMS, INC. React Native Documentation. Disponível em: <https://reactnative.dev/docs/getting-started>. Acesso em: 20 maio 2026.
- EXPO. Expo Documentation. Disponível em: <https://docs.expo.dev/>. Acesso em: 20 maio 2026.
- GITHUB. GitHub Actions Documentation. Disponível em: <https://docs.github.com/en/actions>. Acesso em: 20 maio 2026.